

TECHNICAL REFERENCE

Backend Architecture

Enterprise Multi-Vendor E-Commerce Marketplace

A comprehensive reference for the Node.js/TypeScript backend — covering module structure, authentication, database patterns, background jobs, payments, and more.

Node.js

TypeScript

Express.js

Prisma ORM

PostgreSQL

Redis

BullMQ

Socket.io

Stripe

JWT RS256

Zod

CONTENTS

- 1 Architecture Overview
- 2 Configuration Layer
- 3 Authentication & RBAC
- 4 Database Layer (Prisma)
- 5 Validation Layer (Zod)
- 6 Background Jobs (BullMQ)
- 7 Email Service
- 8 Payment Flow (Stripe)
- 9 Error Handling
- 10 API Response Format

1

Architecture Overview

The backend follows a strict **domain-driven, layered architecture**. Every business domain is a self-contained module under `src/modules/`, and each module exposes the same four-layer stack — keeping routing, HTTP handling, business logic, and data access completely separated.

Module Structure

Each domain directory contains exactly six files following a uniform naming convention:

```
src/modules/{domain}/
├─ {domain}.routes.ts    // Express Router – URL mapping + middleware chain
├─ {domain}.controller.ts // Thin HTTP adapter – calls service, sends response
├─ {domain}.service.ts    // Business logic – orchestrates repository + external calls
├─ {domain}.repository.ts // All Prisma DB calls
├─ {domain}.types.ts      // TypeScript interfaces for the module
└─ {domain}.validator.ts  // Zod schemas for request validation
```

Domain Modules

The following domains are implemented, each as an independent module:

- auth
- users
- products
- categories
- orders
- payments
- vendors
- cart
- coupons
- reviews
- notifications
- analytics
- search

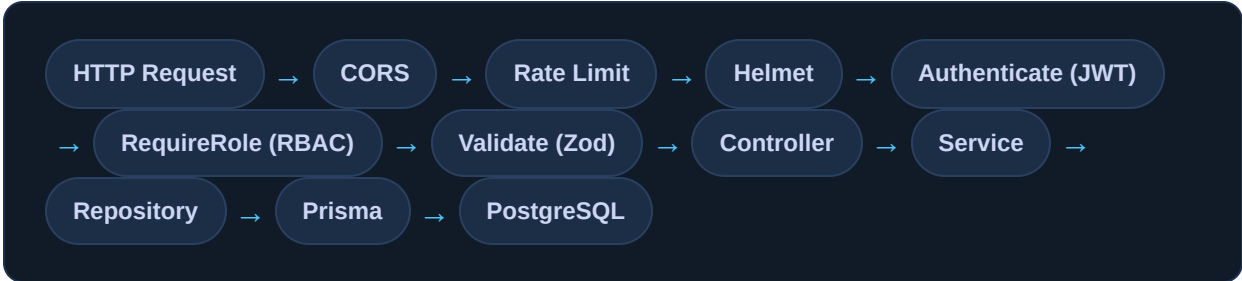
Layer Responsibilities

LAYER	FILE	RESPONSIBILITY
Routes	<code>{domain}.routes.ts</code>	Defines URL patterns, applies middleware chain (auth, RBAC, validation), delegates to controller methods.

LAYER	FILE	RESPONSIBILITY
Controller	{domain}.controller.ts	Thin HTTP adapter. Extracts request data, calls the service, sends the HTTP response. No business logic.
Service	{domain}.service.ts	All business logic lives here. Orchestrates one or more repository calls and external services (email, Stripe, etc.).
Repository	{domain}.repository.ts	Encapsulates all Prisma queries. Services never import Prisma directly — they call repository methods.
Types	{domain}.types.ts	TypeScript interfaces and type aliases used across all layers of the module.
Validator	{domain}.validator.ts	Zod schemas for body and query validation. Consumed by the validate() middleware in routes.

Request Lifecycle

Every inbound HTTP request passes through the following ordered middleware pipeline before reaching the controller:



i Middleware ordering is critical.
Authentication runs before RBAC (you must know *who* before checking *what they can do*).
Validation runs after RBAC to avoid wasting CPU parsing bodies that would be rejected anyway.

2 Configuration Layer (src/configs/)

All runtime infrastructure is initialised in singleton modules under `src/configs/` and imported wherever needed. This prevents multiple client instances from being created across modules.

database.ts — Prisma Client Singleton

Exports a single `PrismaClient` instance shared across the entire application. It also patches `Decimal.prototype.toJSON` to return a JavaScript `float` instead of a string — Prisma's default serialisation wraps `Decimal` values in quotes, which breaks downstream consumers expecting numeric JSON.

● SRC/CONFIGS/DATABASE.TS

```
import { PrismaClient, Prisma } from '@prisma/client';

// Singleton – reuse across hot-reloads in development
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ?? new PrismaClient({ log: ['query', 'error'] });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;

// Fix: Prisma serialises Decimal as string – patch to return float
(Prisma.Decimal.prototype as any).toJSON = function() {
  return parseFloat(this.toString());
};
```



Why this patch matters.

Without it, any DECIMAL database column (prices, totals, discounts) serialises as a string like "19.99" in JSON. The frontend expects a number. This one-liner fixes it globally for every Prisma query in the app.

env.ts — Validated Environment Variables

Uses `dotenv` to load `.env`, then validates every required variable against a Zod schema at startup. If any required variable is missing or malformed, the process exits immediately with a descriptive error — preventing silent misconfiguration at runtime.

● SRC/CONFIGS/ENV.TS (PATTERN)

```
import { z } from 'zod';
import 'dotenv/config';

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url(),
  JWT_ACCESS_TOKEN_PRIVATE_KEY: z.string().min(1),
  JWT_ACCESS_TOKEN_PUBLIC_KEY: z.string().min(1),
  STRIPE_SECRET_KEY: z.string(),
  SMTP_HOST: z.string(),
  NODE_ENV: z.enum(['development', 'production', 'test']),
  // ... all other required vars
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error('Missing env vars:', parsed.error.flatten());
  process.exit(1); // Fail fast
}

export const env = parsed.data;
```

! app.ts — Express App Factory

Assembles and returns the configured Express application. Registers middleware in order:

- **Helmet** — sets secure HTTP response headers
- **CORS** — configured with `X-Session-ID` in `allowedHeaders` to support guest cart tracking via session ID header
- **express.json()** — body parser for JSON payloads
- **Morgan** — HTTP request logger
- **Route modules** — each domain router mounted at its prefix
- **Global error handler** — catches all forwarded errors, returns unified JSON

! **X-Session-ID in CORS allowedHeaders.**

Guest cart items are tracked by a `sessionId` generated on the frontend and sent as the `X-Session-ID` header. Without explicitly whitelisting it in CORS, browsers block the preflight request and cart operations fail for unauthenticated users.

redis.ts — IORedis Singleton

Exports a single `IORedis` instance created from `REDIS_URL`. This instance is passed as the `connection` option to every BullMQ `Queue` and `Worker`, ensuring all background job infrastructure shares the same connection pool.

socket.ts — Socket.io Server

Initialises the Socket.io server and exports a `emitToUser(userId, event, data)` helper. When a client connects and authenticates, it calls `socket.join(userId)` to enter a room named after the user's ID. The helper then targets that room:

● SRC/CONFIGS/SOCKET.TS

```
export function emitToUser(userId: string, event: string, data: unknown) {
  io.to(userId).emit(event, data);
}

// On connect: client joins its own room
io.on('connection', (socket) => {
  const { userId } = socket.handshake.auth;
  if (userId) socket.join(userId);
});
```

logger.ts — Winston Logger

Provides structured logging via Winston. In **production**, outputs newline-delimited JSON for log aggregation tools (Datadog, CloudWatch, etc.). In **development**, outputs colorized, human-readable logs. The same logger instance is used by all modules, ensuring consistent log levels and formats.

JWT RS256 Strategy

Authentication uses **asymmetric JWT (RS256)** rather than the simpler HS256 symmetric approach. The private key signs tokens; the public key verifies them. This is advantageous in a multi-service architecture because verification services only need the public key — they can never issue tokens.

TOKEN	KEY USED	LIFETIME	PURPOSE
Access Token	ACCESS_TOKEN_PRIVATE_KEY (sign) / PUBLIC_KEY (verify)	15 minutes	Sent as Authorization: Bearer <token> on every API request
Refresh Token	Same RS256 key pair	7 days	Used by the frontend to silently obtain a new access token when it expires

auth.service.ts — Core Operations

register()

1. Hash the password with bcrypt using **12 rounds** (cost factor)
2. Create the user record in PostgreSQL via the user repository
3. Sign both access and refresh tokens with the RS256 private key
4. Return `{ accessToken, refreshToken, user }`

login()

1. Find the user by email
2. Compare the submitted password against the stored bcrypt hash with `bcrypt.compare()`
3. Generate and return both tokens

refreshTokens()

1. Verify the refresh token using the RS256 public key
2. If valid and not expired, issue a new access token
3. Optionally rotate the refresh token (recommended)

Password Reset Flow

1. Generate a cryptographically secure random token via `crypto.randomBytes()`

2. Store the **hashed** version in the database alongside an expiry timestamp
3. Email the **raw** token to the user (via BullMQ email queue)
4. On reset: hash the submitted token, look up the record, verify expiry, update the password

✓ **Security note.**

Only the hash is stored in the database. Even if the database is compromised, the attacker cannot use the stored value to reset passwords — they need the raw token that was emailed.

auth.middleware.ts — JWT Verification

● SRC/MODULES/AUTH/AUTH.MIDDLEWARE.TS

```
export const authenticate = (req: Request, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return next(AppError.unauthorized('No token provided'));
  }
  const token = authHeader.slice(7);
  try {
    const payload = jwt.verify(token, env.ACCESS_TOKEN_PUBLIC_KEY, { algorithm: 'RS256' });
    req.user = payload as { id: string; email: string; role: UserRole };
    next();
  } catch {
    next(AppError.unauthorized('Invalid or expired token'));
  }
};
```

rbac.middleware.ts — Role-Based Access Control

The `requireRole(...roles)` factory middleware checks `req.user.role` against the allowed roles array. Returns a **403 Forbidden** if the user's role is not in the list.

● SRC/MODULES/AUTH/RBAC.MIDDLEWARE.TS

```
export const requireRole = (...roles: UserRole[]) => {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!roles.includes(req.user!.role)) {
      return next(AppError.forbidden('Insufficient permissions'));
    }
  };
};
```



```
    next();
  };
};

// Usage in routes:
router.delete('/products/:id', authenticate, requireRole('VENDOR', 'ADMIN'),
```

4

Database Layer (Prisma)

PostgreSQL is the primary data store, accessed exclusively through **Prisma ORM**. The schema is defined in `prisma/schema.prisma` and all queries are issued through typed repository classes.

Key Models & Relations

MODEL	KEY RELATIONS
User	Has one Vendor, many Orders, many CartItems (via Cart), many Reviews
Vendor	Has one Store, many Products
Product	Belongs to Category, Vendor, Store; has many ProductImages, ProductVariants, ProductInventory, ProductTags
Category	Self-referential (parentId) for hierarchical category tree
Order	Has one OrderShippingAddress, many OrderItems, many OrderStatusHistory, many Payment
Cart	Belongs to User <i>or</i> tracked by sessionId for unauthenticated guests; has many CartItems
Payment	Belongs to Order; has one optional PaymentRefund

Repository Pattern

Every domain has a repository class that encapsulates all Prisma calls. Services import only the repository — never Prisma directly. This centralises query logic, makes testing easier (repositories can be mocked), and prevents query sprawl across the codebase.

The product repository's `findAll` method demonstrates the full pattern — dynamic `where` building, parallel count + fetch, and paginated result assembly:

● SRC/MODULES/PRODUCTS/PRODUCTS.REPOSITORY.TS — FINDALL()

```
async findAll(query: ProductListQuery): Promise<PaginatedResult<Product>> {
  const { skip, take, page, pageSize } = parsePaginationQuery(query);
  const categoryIds = query.categoryIds?.split(',');

  const where: Prisma.ProductWhereInput = {
    deletedAt: null, // Soft-delete guard
    status: query.status ?? 'ACTIVE',
    ...(categoryIds && { categoryId: { in: categoryIds } }),
    ...(query.vendorId && { vendorId: query.vendorId }),
    ...(query.isFeatured !== undefined && { isFeatured: query.isFeatured }),
    ...(query.search && {
      OR: [
        { name: { contains: query.search, mode: 'insensitive' } },
        { description: { contains: query.search, mode: 'insensitive' } },
      ]
    }),
  };

  const [products, total] = await Promise.all([
    prisma.product.findMany({ where, skip, take, include: productInclude }),
    prisma.product.count({ where }),
  ]);

  return buildPaginatedResult(products, total, page, pageSize);
}
```

Prisma Transactions

Multi-table writes are wrapped in `prisma.$transaction()` to ensure atomicity. If any operation inside the transaction throws, all changes are rolled back automatically.

The order creation transaction demonstrates three coordinated operations in a single atomic unit:

● SRC/MODULES/ORDERS/ORDERS.SERVICE.TS — CREATEORDER()

```
const order = await prisma.$transaction(async (tx) => {  
  // 1. Create the order record  
  const newOrder = await tx.order.create({ data: { ...orderData } });  
  
  // 2. Reserve inventory (decrement stock for each variant)  
  for (const item of cartItems) {  
    await tx.productVariant.update({  
      where: { id: item.variantId },  
      data: { stock: { decrement: item.quantity } },  
    });  
  }  
  
  // 3. Clear the cart  
  await tx.cartItem.deleteMany({ where: { cartId: cart.id } });  
  
  return newOrder;  
});
```

 Use the transaction client (tx), not the global prisma.

Inside a transaction callback, always use the tx parameter passed by Prisma. Using the global prisma instance inside a transaction bypasses the transaction entirely.

Soft Deletes

Products use a `deletedAt: DateTime?` field rather than physical deletion. This preserves order history, analytics, and audit trails. All queries must include `deletedAt: null` in the `where` clause to filter out soft-deleted records.

Pagination Utilities

Defined in `src/utls/pagination.ts`, these two functions standardise pagination across all list endpoints:

● SRC/UTLS/PAGINATION.TS

```
export function parsePaginationQuery(query: any) {  
  const page = query.page ?? 1;  
  const pageSize = Math.min(query.pageSize ?? 20, 100); // cap at 100  
  return {
```

```

    skip: (page - 1) * pageSize,
    take: pageSize,
    page, pageSize,
  };
}

export function buildPaginatedResult<T>(
  items: T[], total: number, page: number, pageSize: number
) {
  const totalPages = Math.ceil(total / pageSize);
  const hasNextPage = page < totalPages;
  const hasPrevPage = page > 1;
  return { items, meta: { total, page, pageSize, totalPages, hasNextPage, hasPrevPage } };
}

```

j **Page size is capped at 100.**

The `Math.min(..., 100)` guard prevents clients from requesting unbounded result sets, which could exhaust database and memory resources.

5 Validation Layer (Zod)

Every `POST` and `PATCH` request body is validated by a Zod schema before the controller is ever called. Zod schemas live in `{domain}.validator.ts` and are applied via the `validate()` middleware factory.

The `validate()` Middleware

● SRC/MIDDLEWARES/VALIDATE.MIDDLEWARE.TS

```

export function validate(schema: ZodSchema) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return next(
        AppError.validation(result.error.flatten().fieldErrors)
      );
    }
    next();
  };
}

```

```

    );
  }
  req.body = result.data; // Replace with type-coerced, validated data
  next();
};
}

```

Using `safeParse` (rather than `parse`) avoids throwing — errors are forwarded to the global error handler via `next()`. Critically, `req.body` is **replaced** with `result.data`, so the controller always receives fully typed, coerced data.

Query Parameter Transforms

HTTP query parameters arrive as strings. Zod's `.transform()` converts them to the correct types. The pattern must preserve `undefined` to avoid applying filters on every request when the parameter is absent:

● SRC/MODULES/PRODUCTS/PRODUCTS.VALIDATOR.TS — QUERY SCHEMA

```

export const productListQuerySchema = z.object({
  // String → number, preserving undefined
  page: z.string().optional().transform((v) => v !== undefined ? Number(v) : undefined),
  pageSize: z.string().optional().transform((v) => v !== undefined ? Number(v) : undefined),

  // String → boolean, preserving undefined
  isFeatured: z.string().optional().transform((v) => v !== undefined ? v === 'true' : undefined),

  // Plain string fields need no transform
  search: z.string().optional(),
  vendorId: z.string().optional(),
  categoryIds: z.string().optional(), // comma-separated IDs
});

```

⚠ The undefined-preserving transform is critical.

A naive `.transform((v) => v === 'true')` maps undefined to false. This silently applies `isFeatured: false` to every product list request, even when the client didn't send the parameter — filtering out all featured products unexpectedly. Always check `v !== undefined` before transforming.

Body Validation Example

● SRC/MODULES/AUTH/AUTH.VALIDATOR.TS

```
export const registerSchema = z.object({
  email:      z.string().email('Invalid email address'),
  password:   z.string().min(8, 'Password must be at least 8 characters'),
  firstName:  z.string().min(1).max(50),
  lastName:   z.string().min(1).max(50),
  role:       z.enum(['CUSTOMER', 'VENDOR']).default('CUSTOMER'),
});

// Route usage:
router.post('/register', validate(registerSchema), authController.register);
```

6 Background Jobs (BullMQ)

Long-running or side-effect tasks (sending emails, creating in-app notifications) are handled asynchronously via BullMQ queues backed by Redis. This keeps request handlers fast — adding a job to a queue is near-instantaneous.

Queue Definitions

`src/jobs/email.job.ts``src/jobs/notification.job.ts`

Redis Connection

IORedis singleton from

`src/configs/redis.ts`

Passed as `connection` to every Queue and Worker

Email Queue

● SRC/JOB/EMAIL.JOB.TS

```
import { Queue, Worker } from 'bullmq';
import { redis } from '../configs/redis';

export const emailQueue = new Queue('email-queue', { connection: redis });
```

```
new Worker('email-queue', async (job) => {
  switch (job.name) {
    case 'send-order-confirmation':
      await sendEmail({
        to: user.email,
        subject: 'Order Confirmed',
        html: orderConfirmationTemplate(job.data),
      });
      break;
    case 'send-password-reset':
      await sendEmail({
        to: user.email,
        subject: 'Reset Your Password',
        html: passwordResetTemplate(job.data),
      });
      break;
  }
}, { connection: redis });
```

Notification Queue

● SRC/JOBS/NOTIFICATION.JOB.TS

```
export const notificationQueue = new Queue('notification-queue', { connection: redis });

new Worker('notification-queue', async (job) => {
  if (job.name === 'in-app-notification') {
    const { userId, type, title, body, metadata } = job.data;

    // 1. Persist to database
    await prisma.notification.create({
      data: { userId, type, title, body, metadata },
    });

    // 2. Push to connected client via Socket.io
    emitToUser(userId, 'notification:new', { title, body });
  }
}, { connection: redis });
```

Adding Jobs from Services

After order creation in `orders.service.ts`, jobs are enqueued before returning the response:

● SRC/MODULES/ORDERS/ORDERS.SERVICE.TS — POST-ORDER JOB DISPATCH

```
// After prisma.$transaction completes:
await emailQueue.add('send-order-confirmation', {
  orderId: order.id,
  userId: order.userId,
});

await notificationQueue.add('in-app-notification', {
  userId: order.userId,
  type: 'ORDER_PLACED',
  title: 'Order Placed',
  body: `Your order ${order.orderNumber} has been placed`,
  metadata: { orderId: order.id },
});
```

Monitoring BullMQ

METHOD	COMMAND / SETUP
Bull Board (UI)	<code>npm install @bull-board/express @bull-board/api</code> — mounts a dashboard at <code>/admin/queues</code>
Redis CLI — list waiting jobs	<code>redis-cli LRange bull:email-queue:wait 0 -1</code>
Redis CLI — all queue keys	<code>redis-cli KEYS bull:*</code>

7

Email Service

Email delivery uses **Nodemailer** with SMTP transport. Configuration is injected from environment variables: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`.

Emails are never sent synchronously from request handlers — they are always dispatched through the BullMQ email queue.

Testing Locally — Three Options

Option 1 Mailtrap — Recommended for Development

Mailtrap catches all outbound emails in a virtual inbox. No real recipients ever receive test emails.

1. Create a free account at **mailtrap.io**
2. Navigate to Email Testing → Inboxes → SMTP Settings
3. Copy the credentials into your `.env` file:

• .ENV

```
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=2525
SMTP_USER=your_mailtrap_user
SMTP_PASS=your_mailtrap_pass
SMTP_FROM=noreply@marketplace.com
```

Option 2 Ethereum Email — Zero-Signup

Nodemailer can create a temporary test account programmatically. Use the returned URL to preview emails in a browser.

• TEST SETUP SCRIPT

```
const account = await nodemailer.createTestAccount();
// account.smtp.host, account.user, account.pass

const info = await transporter.sendMail({ to: 'test@example.com', ... })
console.log('Preview URL:', nodemailer.getTestMessageUrl(info));
```

Option 3 Local MailHog (Docker)

MailHog runs a local SMTP server and web UI inside Docker. All emails are captured locally.

● TERMINAL

```
# Start MailHog
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog

# .env settings
SMTP_HOST=localhost
SMTP_PORT=1025

# Web UI at http://localhost:8025
```

8 Payment Flow (Stripe)

Payments use the **Stripe PaymentIntents API** with a server-side-first flow. The backend creates the PaymentIntent, returns the `clientSecret` to the frontend, and the frontend uses Stripe.js to confirm the payment — card details never touch the backend.

Files: `src/modules/payments/payments.service.ts`, `payments.routes.ts`, `payments.controller.ts`

Full Payment Flow

1 POST /orders — Create Order + PaymentIntent

The order service creates the order record, then calls `stripe.paymentIntents.create()`. The returned `stripePaymentIntentId` is stored in the `Payment` table. The `clientSecret` is returned to the frontend in the response.

2 Frontend — `stripe.confirmPayment()`

The frontend receives `clientSecret` and calls `stripe.confirmPayment({ clientSecret, ... })` via Stripe.js. Stripe handles 3DS authentication, card validation, and charging directly.

3 POST /payments/confirm — Mark as Paid

After Stripe.js confirms success, the frontend calls this endpoint. The service finds the payment by `stripePaymentIntentId`, marks it `PAID`, and transitions the order to `CONFIRMED`.

4

POST /payments/webhook — Stripe Event Handler

Stripe sends server-to-server events for reliability. The webhook handler uses a **raw body parser**

(not `express.json()`) so `stripe.webhooks.constructEvent()` can verify the signature. Handles `payment_intent.succeeded` and `payment_intent.payment_failed`.

⚠ Webhook route must use raw body middleware.

Mount the webhook route *before* `express.json()` in `app.ts`, or use `express.raw({ type: 'application/json' })` specifically for that route. If the body is already JSON-parsed when Stripe's SDK tries to verify the signature, the verification fails with an invalid signature error.

Mock / Demo Mode

When `STRIPE_SECRET_KEY` is a placeholder (length ≤ 40 chars), `isStripeConfigured()` returns false. In this mode the service skips the Stripe API call entirely:

● SRC/MODULES/PAYMENTS/PAYMENTS.SERVICE.TS — MOCK MODE

```
function isStripeConfigured(): boolean {
  return env.STRIPE_SECRET_KEY.length > 40;
}

if (!isStripeConfigured()) {
  // Return a mock clientSecret — frontend detects 'mock_' prefix
  return {
    clientSecret: `mock_${orderId}`,
    stripePaymentIntentId: `mock_${orderId}`,
  };
}
// Otherwise: real Stripe call
const intent = await stripe.paymentIntents.create({ amount, currency: 'usd' })
```

The frontend detects the `mock_` prefix and renders a "Demo Mode — Simulate Payment" button instead of the Stripe card form.

Error Handling

All errors are centralised through the `AppError` class and a single global Express error handler middleware. This ensures every error produces the same JSON shape regardless of where it originated.

AppError Class

● SRC/MIDDLEWARES/ERROR.MIDDLEWARE.TS — APPERROR

```
class AppError extends Error {
  constructor(
    public message: string,
    public statusCode: number,
    public errors?: unknown
  ) {
    super(message);
    Object.setPrototypeOf(this, AppError.prototype);
  }

  static badRequest(msg: string) { return new AppError(msg, 400); }
  static unauthorized(msg: string) { return new AppError(msg, 401); }
  static forbidden(msg: string) { return new AppError(msg, 403); }
  static notFound(resource: string) { return new AppError(`${resource} not found`, 404); }
  static conflict(msg: string) { return new AppError(msg, 409); }
  static validation(errors: unknown) { return new AppError('Validation failed', 400, errors); }
}
```

Global Error Handler Middleware

Registered as the **last middleware** in `app.ts` (after all routes). Express identifies it as an error handler by its four-argument signature `(err, req, res, next)`. It normalises all thrown errors into a consistent JSON envelope:

● SRC/MIDDLEWARES/ERROR.MIDDLEWARE.TS — HANDLER

```
export const errorHandler = (
  err: unknown, req: Request, res: Response, next: NextFunction
) => {
  if (err instanceof AppError) {
```

```
return res.status(err.statusCode).json({
  success: false,
  message: err.message,
  data: null,
  errors: err.errors ?? null,
});
}
// Unhandled/unexpected errors
logger.error(err);
return res.status(500).json({
  success: false,
  message: 'Internal server error',
  data: null,
  errors: null,
});
};
```

Error Response Shape

● ERROR JSON RESPONSE — EXAMPLE (VALIDATION FAILURE)

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": {
    "email": ["Invalid email address"],
    "password": ["Password must be at least 8 characters"]
  }
}
```

HTTP Status Code Reference

FACTORY METHOD	STATUS	COMMON USE
AppError.badRequest()	400	Invalid input not caught by Zod validation
AppError.validation()	400	Zod schema failure — includes field-level errors
AppError.unauthorized()	401	Missing or invalid JWT token

FACTORY METHOD	STATUS	COMMON USE
<code>AppError.forbidden()</code>	403	Authenticated but insufficient role
<code>AppError.notFound()</code>	404	Resource lookup returned null
<code>AppError.conflict()</code>	409	Duplicate email on register, etc.

10

API Response Format

Every successful API response follows the same JSON envelope shape, produced by `sendSuccess()` and `sendCreated()` utility functions in `src/utls/response.ts`. This contract is relied upon by the frontend API wrapper to uniformly extract data and pagination metadata.

Standard Success Response

● JSON — PAGINATED LIST RESPONSE

```
{
  "success": true,
  "message": "Products fetched",
  "data": [
    { "id": "prod_123", "name": "Wireless Headphones", "price": 49.99 },
    // ...
  ],
  "meta": {
    "total": 26,
    "page": 1,
    "pageSize": 20,
    "totalPages": 2,
    "hasNextPage": true,
    "hasPreviousPage": false
  }
}
```

Response Utility Functions

● SRC/UTILS/RESPONSE.TS

```
export function sendSuccess(
  res: Response, message: string, data: unknown = null, meta?: unknown
) {
  return res.status(200).json({ success: true, message, data, meta: meta ?? null })
}

export function sendCreated(
  res: Response, message: string, data: unknown = null
) {
  return res.status(201).json({ success: true, message, data, meta: null });
}

// Controller usage:
sendSuccess(res, 'Products fetched', result.items, result.meta);
sendCreated(res, 'Product created', newProduct);
```

Frontend API Wrapper Integration

The frontend `api.ts` Axios/fetch wrapper unwraps the envelope automatically. Consumers of the API client receive the inner payload directly — they never deal with the envelope:

● FRONTEND/SRC/LIB/API.TS (PATTERN)

```
// The wrapper unwraps the envelope:
const response = await apiClient.get('/products');
// response.data → the payload array (not the envelope)
// response.meta → pagination metadata
// response.message → human-readable message

// Implementation in api.ts:
axios.interceptors.response.use((response) => {
  const { data, meta, message } = response.data;
  return { ...response, data, meta, message };
});
```

Response Fields Summary

FIELD	TYPE	ALWAYS PRESENT	DESCRIPTION
success	boolean	Yes	true for 2xx responses, false for errors
message	string	Yes	Human-readable description of the outcome
data	T null	Yes	The response payload — object, array, or null
meta	PaginationMeta null	Yes	Pagination info for list endpoints, null for single-resource responses
errors	FieldErrors null	Error responses only	Field-level validation errors from Zod, keyed by field name

✓ **Consistent contract across all endpoints.**

The uniform envelope means the frontend can handle loading states, error display, and pagination with generic utilities — no per-endpoint parsing logic required.